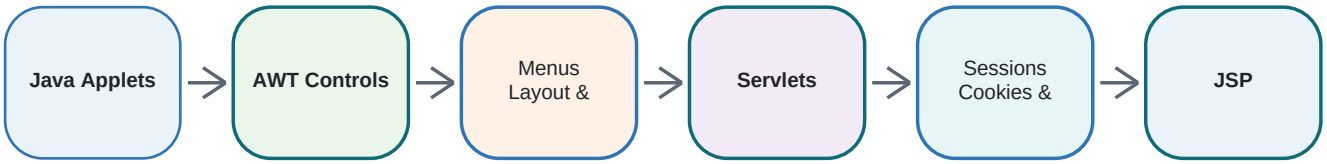
AM513PE: WEB PROGRAMMING

UNIT – IV • APPLETS, AWT, SERVLETS & JSP

R22 B.Tech. CSE (AI & ML) • JNTU Hyderabad

Visual Learning Notes

Designed for classroom study, semester examinations, quick revision and interview-oriented understanding. The unit is organized as **concept** → **diagram** → **example** → **exam point**.



IMPORTANT ACADEMIC NOTE

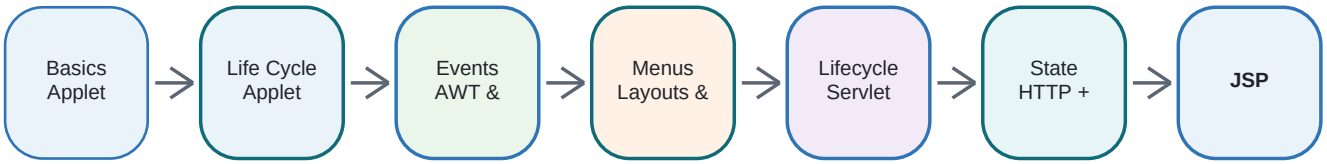
Java Applets are a legacy technology and are no longer supported by modern mainstream browsers. They are included here because they are part of the prescribed syllabus. For modern web development, focus on the underlying concepts and use current web technologies in real projects.

1. Unit IV at a Glance

The syllabus groups Unit IV into two broad areas: **Java GUI/client-side concepts** and **server-side Java web programming**.

Syllabus area	What you should learn	Exam focus
Java Applets	Applet concept, life cycle, images, sound, parameters	Life-cycle methods + parameter passing
AWT	Windows, graphics/text, controls	Controls, event handling, layouts
Layout Managers	Flow, Border, Grid, Card, GridBag concepts	Differences + use cases
Menus	Menu bar, menus, menu items	Menu structure
Servlets	Life cycle, API, HTTP request/response	init/service/destroy + GET/POST
Cookies	Client-side state tracking	Create/read cookies
Session Tracking	Server-side user state	HttpSession
JSP	Dynamic server-side pages	JSP basics + servlet relationship

Unit IV Learning Path



MEMORY TRICK

Applet: init → start → paint → stop → destroy. **Servlet:** init → service → destroy. Think of both as life-cycle based components, but a servlet is server-side.

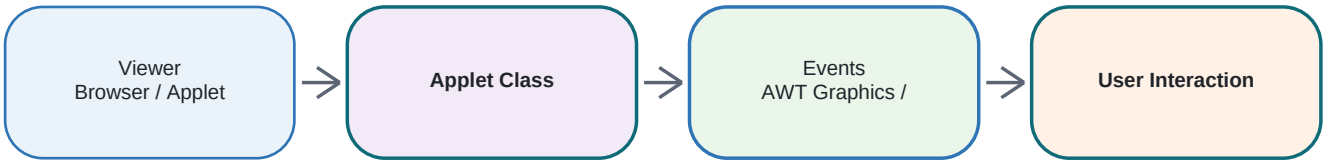
2. Java Applets

A **Java applet** is a small Java program historically designed to execute inside a web browser or an applet viewer. It is different from a standalone Java application because its execution was controlled by an applet environment.

Key characteristics

- Historically used for interactive browser-based Java content.
- Usually extends **java.applet.Applet** or, for Swing-based legacy applets, **javax.swing.JApplet**.
- Does not normally use a traditional **main()** method for its life cycle.
- The applet environment invokes life-cycle methods such as **init()**, **start()**, **stop()** and **destroy()**.
- Can draw using AWT graphics APIs and respond to user interaction.

Applet Execution Model

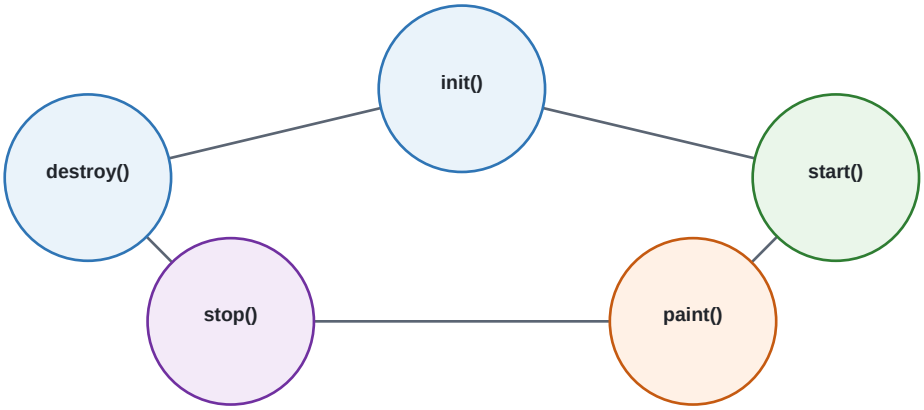


EXAM POINT

The most important applet question is usually the **life cycle**. Be able to write the methods in order and explain what each method does.

3. Applet Life Cycle

The applet life cycle describes how the hosting environment initializes, starts, pauses and finally removes an applet.



Method	When it is called	Purpose
init()	Once, when applet is initialized	Perform one-time initialization such as reading parameters or creating objects.
start()	When applet becomes active	Start/restart activities such as animation or threads.
paint(Graphics g)	When display needs painting	Draw text, shapes or images on the applet surface.
stop()	When applet becomes inactive	Pause activities such as animation or background work.
destroy()	When applet is being unloaded	Release resources and perform final cleanup.

Minimal legacy example

```
import java.applet.Applet;
import java.awt.Graphics;

public class HelloApplet extends Applet {
public void init() {
// one-time initialization
}

public void paint(Graphics g) {
g.drawString("Hello Applet", 40, 50);
}
}
```

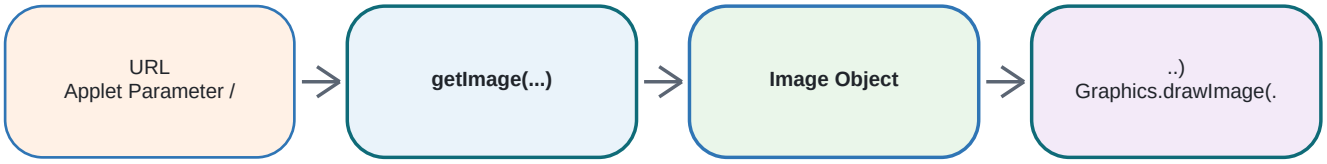
REMEMBER

init() = initialize once • **start()** = begin/resume • **paint()** = display • **stop()** = pause • **destroy()** = cleanup

4. Images, Sound & Parameters in Applets

The syllabus specifically includes adding images and sound, and passing parameters to an applet.

Adding an image — conceptual flow



Passing parameters

Parameters can be supplied by the hosting HTML configuration and read by the applet using **getParameter()**.

```
String name = getParameter("user");
if (name == null) {
name = "Guest";
}
g.drawString("Welcome " + name, 30, 50);
```

Sound concept

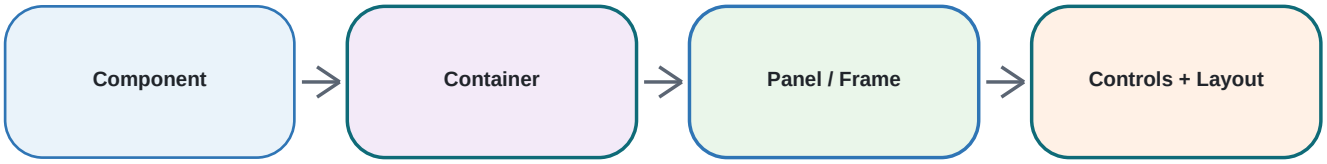
- The legacy applet APIs provided mechanisms for loading and playing audio clips.
- The important examination idea is the sequence: **locate resource** → **load resource** → **play / loop** → **stop when necessary**.
- In modern web applications, browser audio is handled using HTML5 and JavaScript rather than Java applets.

> **MODERN CONTEXT**
>
> Applet-specific APIs are primarily of historical/academic interest. The transferable concepts are resource loading, initialization, rendering, event handling and lifecycle management.

5. AWT — Abstract Window Toolkit

AWT (Abstract Window Toolkit) is Java's original GUI toolkit for creating windows, controls and graphics. It provides components such as Button, Label, TextField, Checkbox and Choice.

AWT component hierarchy — simplified



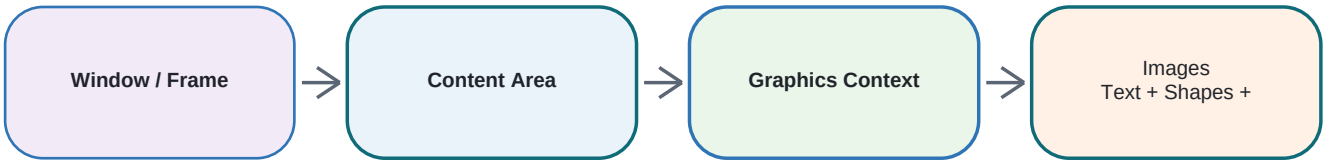
Component	Purpose	Example
Label	Display non-editable text	Label(Name)
Button	Trigger an action	Submit / Login
TextField	Single-line text input	Username
TextArea	Multi-line text input	Comments
Checkbox	On/off selection	Remember me
CheckboxGroup	Radio-button style single choice	Male / Female
Choice	Drop-down selection	Country
List	List of selectable items	Departments
Frame	Top-level window	Application window

EXAM TIP

For theory questions, distinguish **component** (a visible GUI element) from **container** (an object that can hold components). A Frame is a top-level container.

6. Windows, Graphics and Text

AWT can create windows and draw text/shapes using a **Graphics** object. AWT graphics are based on a coordinate system whose origin is normally at the top-left of the drawing area.



Common Graphics operations

Method	Use
drawString()	Draw text at a specified coordinate.
drawLine()	Draw a line between two points.
drawRect() / fillRect()	Draw / fill a rectangle.
drawOval() / fillOval()	Draw / fill an oval.
drawImage()	Render an image.

Example

```
public void paint(Graphics g) {
g.drawString("Web Programming", 30, 40);
g.drawRect(30, 60, 120, 50);
g.drawOval(180, 60, 80, 50);
}
```

COORDINATE TIP

For a typical AWT drawing area, **(0,0)** is near the top-left. Increasing x moves right; increasing y moves down.

7. Event Handling in AWT

GUI programs are event-driven. Instead of continuously asking whether a user clicked a button, the program registers a listener and responds when an event occurs.



Event / Listener	Typical use
ActionEvent / ActionListener	Button click, menu item action
MouseEvent / MouseListener	Mouse click, press, release, enter, exit
KeyEvent / KeyListener	Keyboard activity
WindowEvent / WindowListener	Window open, close, activate, deactivate
ItemEvent / ItemListener	Checkbox or choice selection changes

Button example

```
Button b = new Button("Login");
b.addActionListener(e -> {
System.out.println("Login clicked");
});
```

CORE IDEA

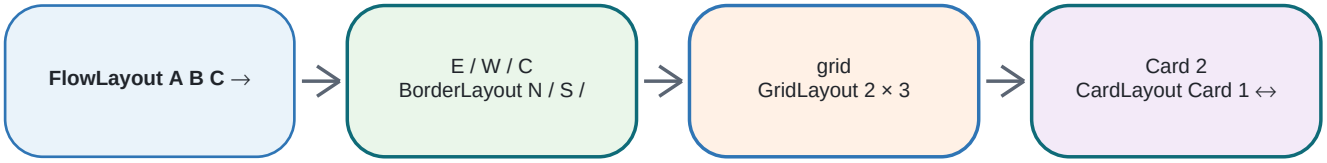
Event source generates the event → listener receives it → callback/handler executes the required action.

8. Layout Managers

A layout manager automatically controls the position and size of GUI components inside a container. This avoids hard-coding every component coordinate.

Layout Manager	Arrangement idea	Typical use
FlowLayout	Left-to-right flow; wraps to next line	Simple rows of controls
BorderLayout	North, South, East, West, Center	Main window sections
GridLayout	Equal-sized rows and columns	Keypads / uniform grids
CardLayout	One component/card visible at a time	Wizard / multi-page UI
GridBagLayout	Flexible grid with constraints	Complex professional forms

Visual comparison

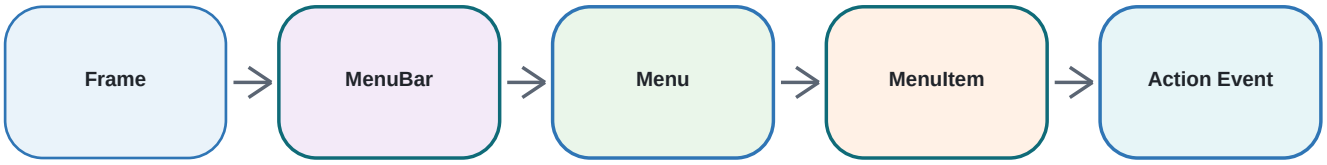


EXAM SHORTCUT

Flow = flow of components • **Border** = five regions • **Grid** = equal cells • **Card** = one card at a time • **GridBag** = flexible constraints.

9. Menus in AWT

Menus organize commands such as File, Edit, View and Help. A typical menu structure contains a **MenuBar**, one or more **Menu** objects and **MenuItem** objects.



Conceptual example

```
MenuBar bar = new MenuBar();
Menu file = new Menu("File");
MenuItem open = new MenuItem("Open");
MenuItem exit = new MenuItem("Exit");

file.add(open);
file.add(exit);
bar.add(file);
frame.setMenuBar(bar);
```

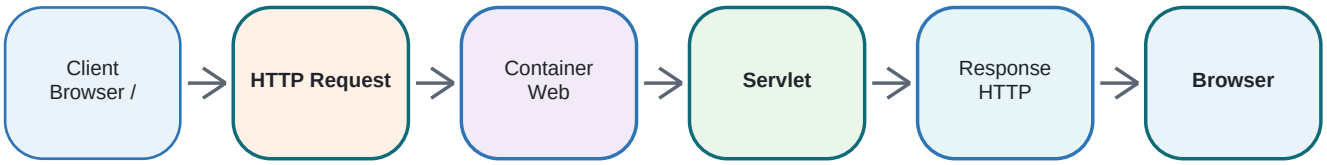
Common menu components

- **MenuBar:** top-level container for menus.
- **Menu:** a drop-down collection of commands.
- **MenuItem:** an individual command.
- **CheckboxMenuItem:** menu command that can be selected/unselected.
- **Separator:** visually groups related menu commands.

10. Servlets — Server-Side Java

A **Servlet** is a Java class that runs on a web/application server and processes client requests, usually HTTP requests. It can generate dynamic responses.

Web request flow



Why servlets?

- Process form submissions and HTTP requests.
- Generate dynamic HTML or other responses.
- Interact with databases and application services.
- Maintain state using cookies, sessions and related mechanisms.
- Run on the server rather than inside the user's browser.

> **MODERN NOTE**
>
> Servlets remain an important foundation of the Java web ecosystem. In modern applications they are commonly used underneath frameworks such as Spring MVC/Spring Boot rather than being written as raw servlets for every endpoint.

11. Servlet Life Cycle

The servlet container manages the servlet's life cycle. The three key methods are **init()**, **service()** and **destroy()**.



Method	Frequency	Purpose
init()	Once	Initialize resources when the servlet is created.
service()	Many times	Process incoming requests and generate responses.
destroy()	Once	Release resources before servlet removal.

HTTP-specific methods

- **doGet()** — handles HTTP GET requests.
- **doPost()** — handles HTTP POST requests.
- Other HttpServlet methods include doPut(), doDelete(), doHead(), doOptions() and doTrace().

Simplified servlet example

```
import jakarta.servlet.http.*;
import jakarta.servlet.annotation.WebServlet;
import java.io.*;

@WebServlet("/hello")
public class HelloServlet extends HttpServlet {
protected void doGet(HttpServletRequest req,
HttpServletResponse resp)
throws IOException {
resp.setContentType("text/html");
resp.getWriter().println("Hello!");
}
}
```

12. Servlet API — Request & Response

The servlet API provides objects through which the servlet reads request data and writes a response.

Object	Important methods / purpose
HttpServletRequest	getParameter(), getParameterNames(), getSession(), getCookies(), getMethod()
HttpServletResponse	setContentType(), setStatus(), addCookie(), getWriter(), sendRedirect()
HttpSession	setAttribute(), getAttribute(), removeAttribute(), invalidate()
Cookie	getName(), getValue(), setMaxAge()

Request → processing → response



Reading a form parameter

```
String username = request.getParameter("username");

response.setContentType("text/html");
PrintWriter out = response.getWriter();
out.println("Welcome " + username + "");
```

SECURITY NOTE

Never directly trust request parameters. Validate input and use safe output/database practices. For database work, prefer parameterized SQL such as PreparedStatement.

13. Handling HTTP Request & Response

HTTP follows a request/response model. The browser sends a request to the server; the server returns a response containing a status, headers and optional body.

Part	Example	Meaning
Method	GET / POST	Action requested by client
URL	/login	Resource/endpoint requested
Headers	Content-Type, Cookie	Metadata about request/response
Status	200, 302, 404, 500	Result of processing
Body	HTML / JSON / text	Actual content transferred

GET vs POST

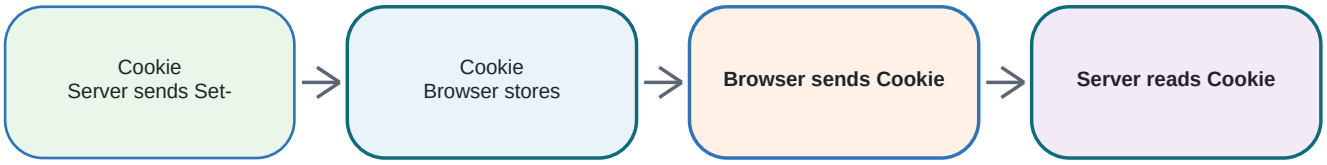
Aspect	GET	POST
Typical use	Retrieve data	Submit/create/process data
Parameters	Often visible in URL query string	Usually carried in request body
Caching/bookmarking	More naturally cacheable/bookmarkable	Not normally treated the same way
Servlet method	doGet()	doPost()

EXAM POINT

A common answer pattern is: **Browser** → **HTTP Request** → **Web Container** → **Servlet** → **Business Logic** → **HTTP Response** → **Browser**.

14. Cookies — Client-Side State Tracking

HTTP is stateless: each request is independent unless an application uses a state-management technique. A **cookie** stores small pieces of data associated with a user's browser.



Create a cookie

```
Cookie c = new Cookie("user", "Naresh");
c.setMaxAge(60 * 60);
response.addCookie(c);
```

Read cookies

```
Cookie[] cookies = request.getCookies();

if (cookies != null) {
for (Cookie c : cookies) {
if ("user".equals(c.getName())) {
String value = c.getValue();
}
}
}
```

COOKIE LIMIT

Cookies are stored on the client side and are sent back with applicable requests. Do not place sensitive information directly into a cookie merely because it is convenient.

15. Session Tracking

A session represents a user's interaction with a web application across multiple requests. Java web applications commonly use **HttpSession** for server-side session data.



Basic HttpSession example

```
HttpSession session = request.getSession();
session.setAttribute("username", "Naresh");

// Later request:
String user = (String) session.getAttribute("username");

// Logout:
session.invalidate();
```

Cookie vs Session

Feature	Cookie	HttpSession
Primary storage	Client/browser	Server-side application
Accessed through	HttpServletRequest / HttpServletResponse	HttpServletRequest.getSession()
Typical use	Preferences, identifiers, small state	Login state, user-specific server data
Security consideration	Client can inspect/manage it	Server keeps application data, but session ID must still be protected

16. Introduction to JSP

JSP (JavaServer Pages) is a server-side technology for creating dynamic web pages using HTML with server-side Java-based constructs. A JSP page is translated/compiled into a servlet by the JSP container.



Why JSP?

- Makes it easier to place dynamic content within an HTML-oriented page.
- Separates page presentation from lower-level servlet request handling.
- Supports JSP directives, actions, expression language and tag libraries.
- Historically used widely in Java web applications; modern applications often use template engines or front-end frameworks depending on architecture.

Simple JSP example

```
<%@ page contentType="text/html;charset=UTF-8" %>


Welcome to Web Programming
Server time: ${pageContext.request.requestURI}
```

KEY RELATIONSHIP

A useful exam statement is: **JSP is ultimately processed on the server and translated into a servlet-based execution model.**

17. JSP Basics — Elements & Flow

JSP element / concept	Purpose	Example
Directive	Provides page-level instructions	<%@ page ... %>
Expression Language (EL)	Access values/objects conveniently	${user.name}
JSP action	Standard action element	
Scriptlet (legacy)	Embeds Java statements	<% ... %>
Expression (legacy)	Outputs a Java expression	<%= value %>

Servlet vs JSP

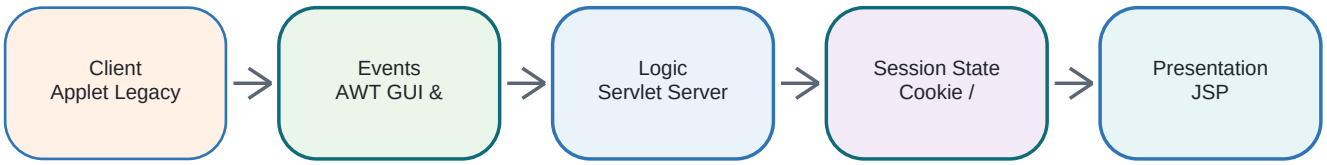
Aspect	Servlet	JSP
Primary role	Request processing / controller logic	Presentation / dynamic page
Style	Java class	HTML-oriented page with server-side constructs
Lifecycle	init/service/destroy	Translated/compiled and then servlet lifecycle applies
Typical use	Process requests, call services	Render dynamic web content

BEST PRACTICE

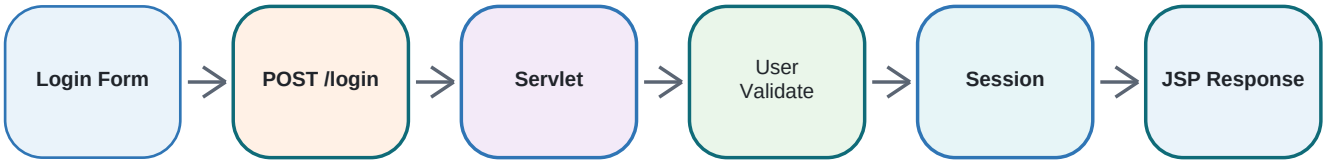
Avoid putting large amounts of business logic directly inside JSP pages. Keep business logic in Java classes/services and use JSP mainly for presentation when using this legacy technology.

18. Putting Unit IV Together

The unit can be understood as a progression from legacy Java client-side programming to server-side Java web programming.



Example: Login application



- The browser submits credentials using HTTP.
- The servlet receives the request and reads parameters.
- The application validates the credentials.
- A session can remember the authenticated user across requests.
- A JSP page can render the resulting user interface.

ARCHITECTURE MEMORY

Request handling → **business processing** → **state management** → **presentation.** This is the conceptual bridge between servlets, sessions and JSP.

19. Important Semester Examination Questions

1. Explain the life cycle of a Java applet.

Answer points: init(), start(), paint(), stop() and destroy(); explain when each is invoked.

2. What is AWT? Explain its important components.

Answer points: Define AWT and discuss Label, Button, TextField, TextArea, Checkbox, Choice, List and Frame.

3. Explain event handling in AWT.

Answer points: Describe event source, event object, listener and handler/callback.

4. Explain different layout managers.

Answer points: FlowLayout, BorderLayout, GridLayout, CardLayout and GridBagLayout with use cases.

5. Explain the life cycle of a servlet.

Answer points: init() once, service() per request, destroy() once.

6. Explain HTTP GET and POST in servlets.

Answer points: Relate GET to doGet() and POST to doPost(); discuss request parameters and response.

7. Explain cookies and session tracking.

Answer points: Compare client-side cookies with server-side HttpSession.

8. Explain HttpServletRequest and HttpServletResponse.

Answer points: Show how request data is read and how response content/status/headers are generated.

9. What is JSP? How is JSP related to servlets?

Answer points: Explain server-side dynamic pages and JSP translation/execution through servlet technology.

10. Differentiate Servlet and JSP.

Answer points: Compare role, coding style, lifecycle and typical usage.

20. One-Page Quick Revision

Concept	Remember this
Applet	Legacy browser Java component; lifecycle: init → start → paint → stop → destroy
AWT	Java GUI toolkit: components, containers, graphics and events
Event handling	Source → event → listener → handler
FlowLayout	Components flow left-to-right and wrap
BorderLayout	North, South, East, West, Center
GridLayout	Equal-sized rows × columns
CardLayout	One card visible at a time
Servlet	Server-side Java component for HTTP request processing
Servlet lifecycle	init → service → destroy
doGet()	Handles GET
doPost()	Handles POST
Cookie	Small client-side state data sent with applicable requests
HttpSession	Server-side session state
JSP	Server-side dynamic page technology; translated into servlet-based execution

LAST-MINUTE MEMORY MAP

Applet: init → start → paint → stop → destroy
AWT: Components + Containers + Events + Layouts
Servlet: Request → service/doGet/doPost → Response
State: Cookie / Session
JSP: Presentation → translated to servlet execution model

End of Unit IV Notes

Use these notes together with classroom examples, prescribed textbooks and previous university question papers. For current application development, treat applet-specific browser deployment as historical material and focus on transferable Java web concepts.